

# Proyecto OLAP de Restaurantes

Documentacion tecnica y evidencias de ejecucion

## **Bases de Datos II**

Pipeline OLAP con Docker, Airflow, PostgreSQL, Spark, Neo4j, Elasticsearch y Metabase

Fecha de evidencias: 16/06/2026

## 1. Resumen ejecutivo

Este documento describe la solución OLAP implementada para un escenario de restaurantes. El proyecto automatiza un flujo completo de analítica: generación de datos operacionales, construcción de un Data Warehouse en PostgreSQL, ejecución de análisis con Spark/PySpark, carga de grafo en Neo4j, asignación de rutas de entrega e indexación de productos en Elasticsearch. La orquestación se realiza mediante un DAG de Airflow ejecutado dentro de Docker Compose.

Las evidencias recopiladas muestran el pipeline completo en estado exitoso, con 2000 órdenes cargados, 19 tablas/vistas en PostgreSQL, 30 productos indexados en Elasticsearch, resultados analíticos generados por Spark y nodos/relaciones consultables en Neo4j.

| Criterio             | Resultado verificado                                                                       |
|----------------------|--------------------------------------------------------------------------------------------|
| Contenedores activos | Servicios principales en estado Up mediante docker compose ps.                             |
| Airflow              | DAG restaurantes_olap_pipeline ejecutado con todas las tareas en verde.                    |
| PostgreSQL           | Tablas operacionales, dimensiones, hechos y vistas OLAP creadas.                           |
| Datos                | Tabla orders con 2000 registros.                                                           |
| Spark                | Análisis de producto popular, horarios pico y crecimiento mensual completado exitosamente. |
| Elasticsearch        | Índice productos con 30 documentos.                                                        |
| Neo4j                | Nodos User, Product, Order, Location y relaciones MADE, CONTAINS, RECOMMENDED, ROUTE_TO.   |

## 2. Objetivos y alcance

### 2.1 Objetivo general

Implementar y documentar un pipeline OLAP dockerizado que permita transformar datos operacionales de restaurantes en información analítica, consultable y visualizable mediante tecnologías de bases de datos relacionales, grafos, búsqueda e inteligencia de negocios.

### 2.2 Objetivos específicos

- Orquestrar el flujo mediante Apache Airflow.
- Construir un esquema dimensional en PostgreSQL con tablas de dimensiones, tablas de hechos y vistas OLAP.
- Ejecutar procesamiento analítico usando PySpark y conexión JDBC hacia PostgreSQL.
- Representar relaciones de usuarios, órdenes, productos y rutas en Neo4j.
- Indexar el catálogo de productos en Elasticsearch para consultas rápidas.
- Dejar evidencias reproducibles mediante comandos, capturas y consultas de validación.

## 3. Tecnologías utilizadas

| Tecnología             | Uso dentro del proyecto                                                   |
|------------------------|---------------------------------------------------------------------------|
| Docker Compose         | Levanta y conecta todos los servicios del ecosistema en una misma red.    |
| Apache Airflow         | Orquesta el pipeline OLAP por medio del DAG restaurantes_olap_pipeline.   |
| PostgreSQL             | Almacena datos operacionales, dimensiones, hechos y vistas OLAP.          |
| Apache Spark / PySpark | Procesa datos de PostgreSQL mediante JDBC y genera resultados analíticos. |

|               |                                                                                  |
|---------------|----------------------------------------------------------------------------------|
| Neo4j         | Modela relaciones entre usuarios, ordenes, productos y rutas de entrega.         |
| Elasticsearch | Indexa productos para conteo y busqueda del catalogo.                            |
| Metabase      | Permite construir visualizaciones y consultas sobre PostgreSQL.                  |
| Python        | Implementa generacion de datos, carga DW, analisis, carga de grafo e indexacion. |

Referencia tecnica: Docker Compose define aplicaciones multi-contenedor mediante servicios, redes y volumenes; Airflow organiza flujos como DAGs con tareas y dependencias; Spark permite leer bases externas mediante JDBC; Neo4j utiliza patrones Cypher MATCH para consultar grafos; Elasticsearch expone APIs REST como `_count` y `_search` para indices.

### 4. Arquitectura general

La solucion se ejecuta como un conjunto de servicios dockerizados conectados por Docker Compose. Airflow funciona como punto de orquestacion y dispara operaciones sobre PostgreSQL, Spark, Neo4j y Elasticsearch.

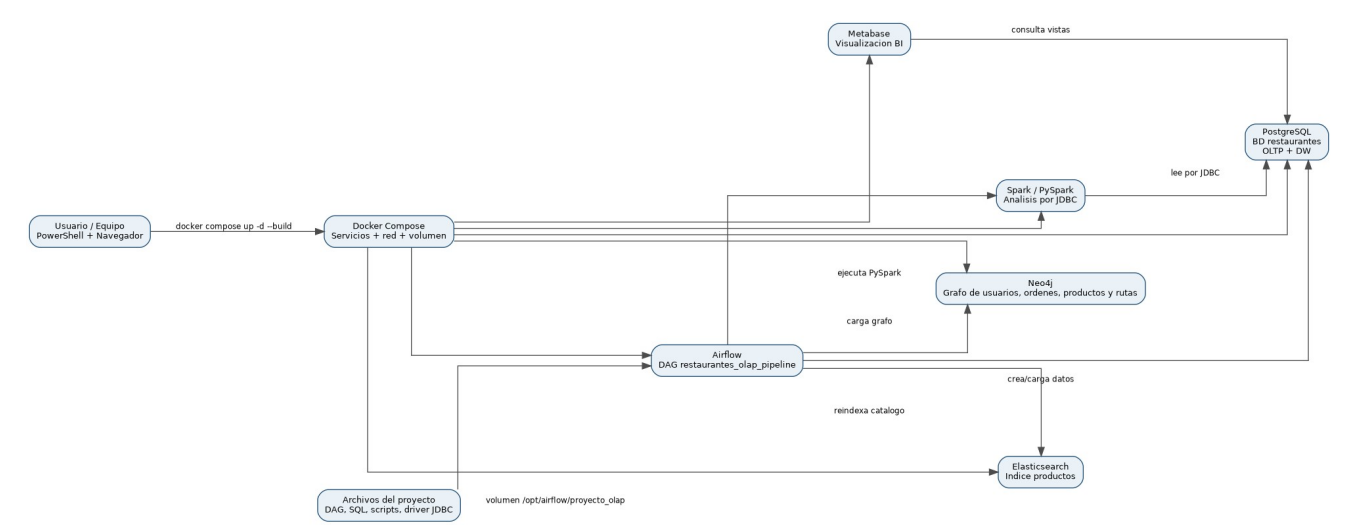


Figura 1. Arquitectura general de servicios dockerizados.

#### 4.1 Endpoints y puertos de servicios

| Servicio             | Endpoint/Puerto           | Funcion                                           |
|----------------------|---------------------------|---------------------------------------------------|
| Airflow              | http://localhost:8080     | Interfaz para consultar y ejecutar el DAG.        |
| PostgreSQL           | localhost:5432            | Base de datos restaurantes. Usuario: admin.       |
| Neo4j Browser        | http://localhost:7474     | Interfaz visual del grafo.                        |
| Neo4j Bolt           | bolt://localhost:7687     | Conexion usada por scripts Python y cypher-shell. |
| Elasticsearch        | http://localhost:9200     | API REST para indice productos.                   |
| Metabase             | http://localhost:3000     | Herramienta BI para dashboards.                   |
| Spark Master UI      | http://localhost:8081     | Interfaz del master Spark.                        |
| Spark Master interno | spark://spark-master:7077 | URL usada por workers de Spark dentro de Docker.  |
| Hive Server          | localhost:10000           | Servicio disponible para integraciones SQL/Hive.  |

## 4.2 Estructura del repositorio

```
proyecto_olap/
├── airflow/dags/restaurantes_olap_dag.py
├── data_generation/
├── docker/airflow/Dockerfile
├── documentacion/capturas/
├── documentacion/consultas_metabase.sql
├── documentacion/consultas_neo4j.cypher
├── drivers/postgresql-42.7.3.jar
├── cargar_elasticsearch.py
├── docker-compose.yml
├── dw_schema.sql
├── ruteo_vecino_cercano.py
└── spark_analisis.py
```

## 5. Configuración Docker y Airflow personalizado

El servicio de Airflow se construye con un Dockerfile propio para instalar Java, PySpark y librerías Python requeridas. Esto evita depender solamente de la imagen base apache/airflow:2.8.1 y permite ejecutar Spark desde el contenedor de Airflow.

| Elemento             | Configuración                                              |
|----------------------|------------------------------------------------------------|
| Imagen base          | apache/airflow:2.8.1                                       |
| Paquetes del sistema | openjdk-17-jre-headless, procps                            |
| Paquetes Python      | pyspark==3.5.3, psycpg2-binary, requests, neo4j, haversine |
| Variables Spark      | JAVA_HOME, PYSPARK_PYTHON, PYSPARK_DRIVER_PYTHON           |
| Volumen del proyecto | ./:/opt/airflow/proyecto_olap                              |

### 5.1 Comandos de despliegue

```
cd proyecto_olap
docker compose up -d --build
```

```
docker compose ps
```

### 5.2 Comandos de mantenimiento

```
docker compose down --remove-orphans
docker compose build --no-cache airflow
docker compose up -d --force-recreate
```

## 6. Pipeline Airflow

El DAG restaurantes\_olap\_pipeline define el orden de ejecución del flujo. Airflow permite modelar el pipeline como tareas dependientes, de modo que la carga del Data Warehouse ocurre antes de los análisis y de la integración con Neo4j y Elasticsearch.

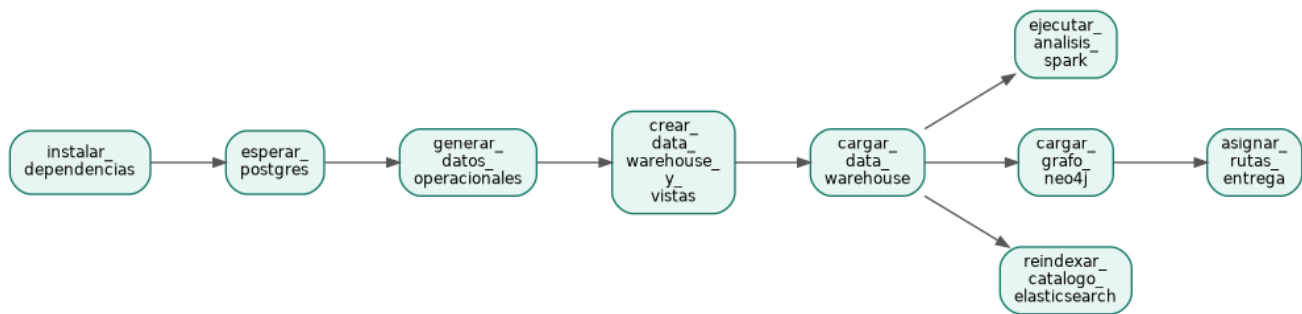


Figura 2. Dependencias del DAG restaurantes\_olap\_pipeline.

| Orden | Tarea                            | Tipo           | Descripcion                                                                                 |
|-------|----------------------------------|----------------|---------------------------------------------------------------------------------------------|
| 1     | instalar_dependencias            | BashOperator   | Instala dependencias Python necesarias dentro del entorno Airflow.                          |
| 2     | esperar_postgres                 | PythonOperator | Valida disponibilidad de PostgreSQL antes de continuar.                                     |
| 3     | generar_datos_operacionales      | PythonOperator | Crea tablas fuente y genera usuarios, productos, ordenes, detalles, reservas y ubicaciones. |
| 4     | crear_data_warehouse_y_vistas    | PythonOperator | Ejecuta dw_schema.sql para crear dimensiones, hechos y vistas OLAP.                         |
| 5     | cargar_data_warehouse            | PythonOperator | Carga dimensiones y hechos desde las tablas fuente.                                         |
| 6     | ejecutar_analisis_spark          | PythonOperator | Ejecuta spark_analisis.py con PySpark.                                                      |
| 7     | cargar_grafo_neo4j               | PythonOperator | Carga nodos y relaciones en Neo4j.                                                          |
| 8     | asignar_rutas_entrega            | PythonOperator | Calcula ruta de entrega por heurística de vecino mas cercano.                               |
| 9     | reindexar_catalogo_elasticsearch | PythonOperator | Crea/recrea indice productos en Elasticsearch.                                              |

docker compose exec airflow airflow dags test restaurantes\_olap\_pipeline 2026-06-16

| Tabla                    |           | Rol                                                  |
|--------------------------|-----------|------------------------------------------------------|
| users                    |           | Clientes con ciudad y coordenadas.                   |
| products                 |           | Catalogo de productos con categoria y precio.        |
| orders                   |           | Pedidos realizados por usuarios, con fecha y estado. |
| order_details            |           | Detalle de productos y cantidades por pedido.        |
| reservations             |           | Reservas realizadas por usuarios.                    |
| locations                |           | Ubicaciones base para analisis geografico.           |
| Objeto                   | Tipo      | Uso analítico                                        |
| dim_date                 | Dimension | Analisis por fecha, mes, dia y dia de semana.        |
| dim_product              | Dimension | Analisis por producto, categoria y precio.           |
| dim_user                 | Dimension | Analisis por cliente y ciudad.                       |
| dim_location             | Dimension | Analisis geografico.                                 |
| fact_sales               | Hecho     | Ventas: cantidad, precio unitario, total y estado.   |
| fact_reservations        | Hecho     | Reservas: cantidad de invitados y estado.            |
| Vista                    |           | Proposito                                            |
| v_ingresos_mes_categoria |           | Ingresos por mes y categoria de producto.            |

|                                     |                                                             |
|-------------------------------------|-------------------------------------------------------------|
| v_actividad_clientes_zona           | Actividad por ciudad/zona: pedidos, reservas e ingresos.    |
| v_pedidos_completados_vs_cancelados | Comparacion mensual entre pedidos completados y cancelados. |
| v_horarios_pico                     | Horas con mayor cantidad de pedidos.                        |
| v_frecuencia_producto_cliente       | Frecuencia de compra de productos por cliente.              |
| v_crecimiento_mensual               | Crecimiento de pedidos e ingresos entre meses.              |
| v_tendencias_consumo                | Tendencias de consumo por categoria.                        |

## 7. Modelo de datos y Data Warehouse

Esta seccion documenta el modelo de datos usado por el proyecto con mas detalle. El diseno separa la capa operacional simulada del Data Warehouse analitico. La capa operacional representa eventos transaccionales, mientras que el Data Warehouse organiza esos datos en dimensiones, hechos y vistas OLAP. Este enfoque facilita consultas agregadas por fecha, producto, cliente, ubicacion, estado de pedido y categoria.

El modelo se puede leer como un esquema estrella: las tablas de hechos contienen las medidas numericas del negocio y se conectan con tablas de dimensiones mediante llaves primarias y foraneas. En este proyecto, fact\_sales concentra la actividad de ventas y fact\_reservations deja preparada la consolidacion de reservas.

### 7.1 Diagrama ampliado del modelo dimensional

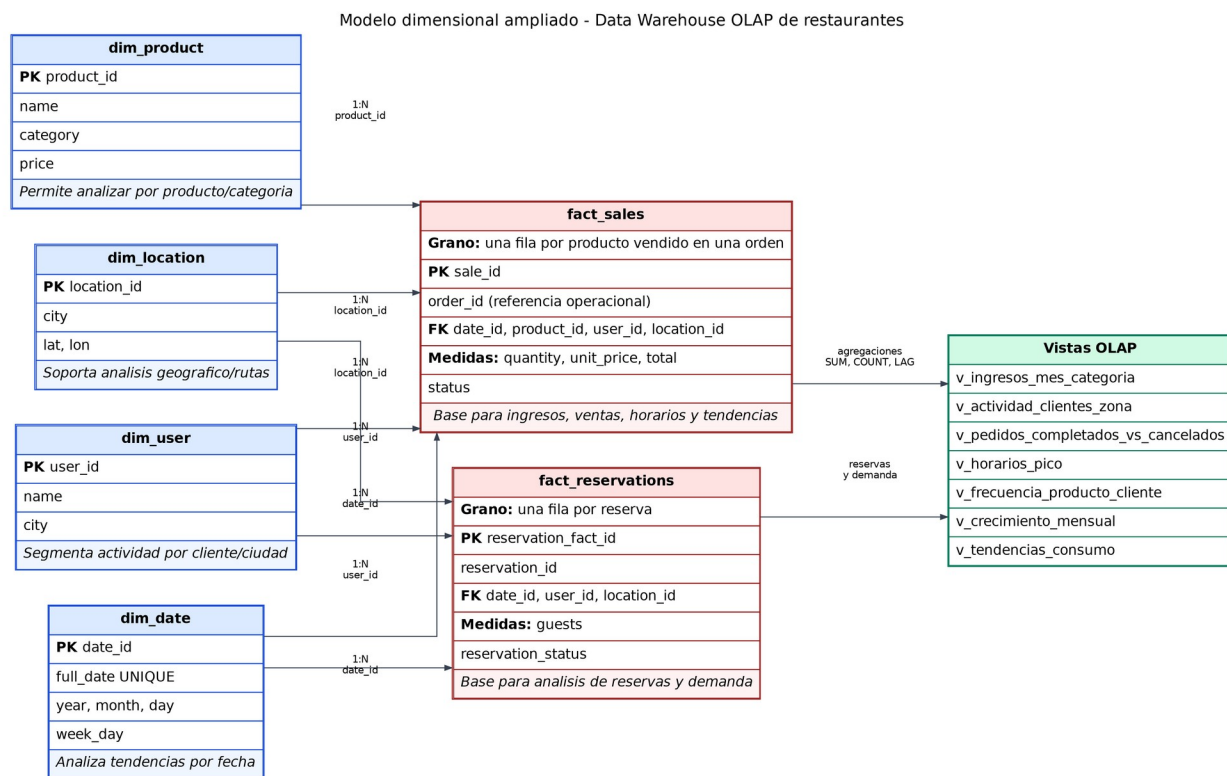


Figura 3. Modelo dimensional ampliado con dimensiones, hechos, grano, llaves y vistas OLAP.

Lectura del diagrama: las dimensiones se muestran en azul porque describen entidades de analisis; las tablas de hechos se muestran en rojo porque almacenan eventos medibles; las vistas OLAP se muestran en verde porque agrupan y resumen los datos para consultas de negocio. Las relaciones 1:N indican que una dimension puede participar en muchas filas de hechos.

| Elemento    | Interpretacion en el diagrama                   | Importancia en OLAP                                                |
|-------------|-------------------------------------------------|--------------------------------------------------------------------|
| Dimensiones | dim_date, dim_product, dim_user y dim_location. | Permiten filtrar, agrupar y segmentar los indicadores.             |
| Hechos      | fact_sales y fact_reservations.                 | Guardan eventos medibles: ventas, cantidades, totales y reservas.  |
| Grano       | Nivel de detalle de cada tabla de hechos.       | Define que significa una fila y evita duplicaciones al consultar.  |
| Vistas OLAP | Consultas agregadas sobre hechos y dimensiones. | Entregan resultados listos para Metabase, reportes y validaciones. |

## 7.2 Capa operacional de origen

La capa operacional contiene las tablas simuladas del sistema transaccional de restaurantes. Estas tablas no estan optimizadas para analitica, sino para representar operaciones como pedidos, lineas de detalle, usuarios, productos, reservas y ubicaciones.

| Tabla operacional | Llave o campo principal | Rol en el sistema fuente                                                | Relacion principal                                   |
|-------------------|-------------------------|-------------------------------------------------------------------------|------------------------------------------------------|
| users             | user_id                 | Clientes que realizan pedidos o reservas. Incluye ciudad y coordenadas. | Se relaciona con orders y reservations.              |
| products          | product_id              | Catalogo de productos con categoria y precio.                           | Se relaciona con order_details y Elasticsearch.      |
| orders            | order_id                | Encabezado del pedido: usuario, fecha y estado.                         | Se relaciona con users y order_details.              |
| order_details     | order_id + product_id   | Detalle de productos comprados por pedido y cantidades.                 | Une orders con products.                             |
| reservations      | reservation_id          | Reservas realizadas por usuarios.                                       | Se relaciona con users y alimenta fact_reservations. |
| locations         | location_id             | Ubicaciones geograficas de referencia.                                  | Se usa para dim_location y analisis espacial.        |

## 7.3 Data Warehouse dimensional

El Data Warehouse se construye en PostgreSQL mediante dw\_schema.sql. Primero se crean las dimensiones, luego las tablas de hechos y finalmente las vistas OLAP. Las dimensiones contienen atributos descriptivos y las tablas de hechos almacenan las metricas de negocio.

| Tabla DW          | Tipo      | Llave principal     | Campos principales                                 | Uso analitico                                      |
|-------------------|-----------|---------------------|----------------------------------------------------|----------------------------------------------------|
| dim_date          | Dimension | date_id             | full_date, year, month, day, week_day              | Analisis temporal: meses, dias y crecimiento.      |
| dim_product       | Dimension | product_id          | name, category, price                              | Analisis por producto y categoria.                 |
| dim_user          | Dimension | user_id             | name, city                                         | Analisis por cliente y zona.                       |
| dim_location      | Dimension | location_id         | city, lat, lon                                     | Analisis geografico y soporte de rutas.            |
| fact_sales        | Hecho     | sale_id             | order_id, FKs, quantity, unit_price, total, status | Ingresos, unidades vendidas, pedidos y tendencias. |
| fact_reservations | Hecho     | reservation_fact_id | reservation_id, FKs, guests, reservation_status    | Reservas, demanda y actividad de clientes.         |

## 7.4 Grano del modelo y reglas de carga

El grano es una de las decisiones mas importantes del modelo porque define el nivel exacto de detalle de cada fila en una tabla de hechos. En este proyecto se definieron dos granos principales:

| Tabla de hechos   | Grano definido                                     | Ejemplo de fila                                                       | Por que es importante                                                           |
|-------------------|----------------------------------------------------|-----------------------------------------------------------------------|---------------------------------------------------------------------------------|
| fact_sales        | Una fila por producto vendido dentro de una orden. | Si una orden contiene tres productos, se generan tres filas de venta. | Permite medir unidades, ingresos y popularidad por producto sin perder detalle. |
| fact_reservations | Una fila por reserva.                              | Cada reserva conserva usuario, fecha, ubicacion,                      | Permite comparar demanda de reservas contra ventas y                            |

|  |  |                     |        |
|--|--|---------------------|--------|
|  |  | invitados y estado. | zonas. |
|--|--|---------------------|--------|

### 7.5 Vistas OLAP y preguntas de negocio

Las vistas OLAP encapsulan consultas SQL frecuentes y evitan repetir joins y agregaciones. En PostgreSQL, una vista define una consulta reutilizable; cuando se consulta la vista, PostgreSQL ejecuta la consulta definida sobre las tablas base.

| Vista OLAP                          | Pregunta que responde                                 | Tablas principales usadas                 | Metricas o calculos                                  |
|-------------------------------------|-------------------------------------------------------|-------------------------------------------|------------------------------------------------------|
| v_ingresos_mes_categoria            | Cuanto ingreso genera cada categoria por mes?         | fact_sales, dim_date, dim_product         | SUM(total) por year, month y category.               |
| v_actividad_clientes_zona           | Que zonas concentran mas actividad?                   | dim_user, fact_sales, users, reservations | Pedidos, reservas e ingresos por city/zone.          |
| v_pedidos_completados_vs_cancelados | Como se comportan los estados de pedidos por mes?     | fact_sales, dim_date                      | COUNT(DISTINCT order_id) por status.                 |
| v_horarios_pico                     | A que hora hay mas pedidos?                           | orders, order_details, products           | COUNT pedidos y SUM ingresos por hora.               |
| v_frecuencia_producto_cliente       | Que productos compra mas cada cliente?                | fact_sales, dim_user, dim_product         | Veces comprado y unidades.                           |
| v_crecimiento_mensual               | Como cambia el volumen de pedidos e ingresos por mes? | fact_sales, dim_date                      | LAG para comparar contra el mes anterior.            |
| v_tendencias_consumo                | Que categorias tienen mayor consumo?                  | fact_sales, dim_product                   | Pedidos, unidades vendidas e ingresos por categoria. |

### 7.6 Flujo de datos y consumo analitico

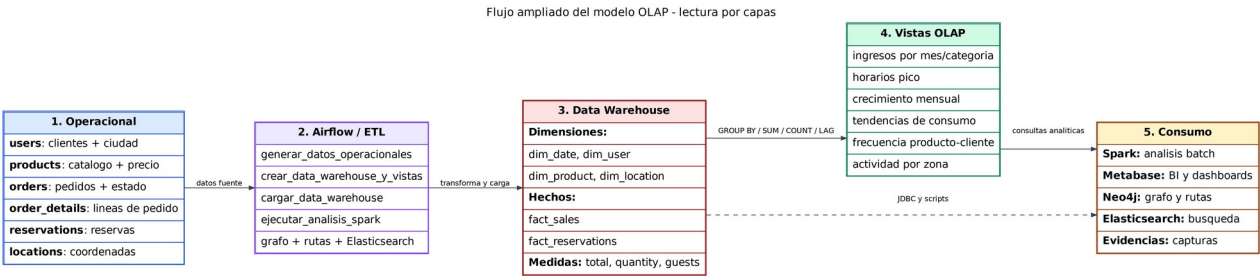


Figura 4. Flujo ampliado por capas desde tablas operacionales hasta consumo analítico.

Este flujo muestra como los datos pasan de una estructura operacional a una estructura analitica. Airflow coordina el orden de ejecucion, PostgreSQL almacena el modelo dimensional, Spark ejecuta analisis batch, Metabase consume consultas BI, Neo4j representa relaciones de usuarios, pedidos y productos, y Elasticsearch permite busquedas sobre el catalogo.

### 7.7 Validaciones del modelo

| Validacion                    | Comando o evidencia                                 | Resultado esperado                                    |
|-------------------------------|-----------------------------------------------------|-------------------------------------------------------|
| Existencia de tablas y vistas | SELECT table_name FROM information_schema.tables... | 19 tablas/vistas en public.                           |
| Cantidad de pedidos fuente    | SELECT COUNT(*) FROM orders;                        | 2000 ordenes.                                         |
| Resultados Spark              | ls /opt/airflow/proyecto_olap/resultados_spark      | crecimiento_mensual, horarios_pico, top_producto_mes. |
| Indexacion Elasticsearch      | GET /productos/_count                               | 30 documentos.                                        |
| Grafo Neo4j                   | MATCH (n) RETURN labels(n), count(n)                | User, Product, Order y Location cargados.             |

## 8. Analisis Spark/PySpark

El archivo spark\_analisis.py crea una SparkSession, lee tablas de PostgreSQL mediante JDBC y ejecuta transformaciones sobre DataFrames. El driver postgresql-42.7.3.jar se mantiene en la carpeta drivers para permitir la conexion desde Spark hacia PostgreSQL.



| Analisis                     | Descripcion                                                                                                         | Salida                               |
|------------------------------|---------------------------------------------------------------------------------------------------------------------|--------------------------------------|
| Producto mas popular por mes | Une orders, order_details y products; filtra pedidos completed; agrupa por mes y producto; usa ranking por ventana. | resultados_spark/top_producto_mes    |
| Horarios pico                | Extrae la hora de order_date y cuenta pedidos por hora.                                                             | resultados_spark/horarios_pico       |
| Crecimiento mensual          | Cuenta pedidos por mes y calcula variacion respecto al mes anterior usando lag.                                     | resultados_spark/crecimiento_mensual |

```
docker compose exec airflow python /opt/airflow/proyecto_olap/spark_analisis.py
```

```
docker compose exec airflow ls /opt/airflow/proyecto_olap/resultados_spark
```

## 9. Grafo Neo4j

Neo4j se utiliza para representar entidades y relaciones que son mas naturales como grafo: usuarios que realizan ordenes, ordenes que contienen productos, recomendaciones entre usuarios y rutas desde el restaurante central hacia usuarios.

| Nodo/Relacion | Cantidad verificada | Descripcion                                                  |
|---------------|---------------------|--------------------------------------------------------------|
| User          | 100                 | Clientes generados con nombre, correo, ciudad y coordenadas. |
| Product       | 30                  | Productos del catalogo.                                      |
| Order         | 2000                | Ordenes generadas.                                           |
| Location      | 1                   | Restaurante Central.                                         |
| MADE          | 2000                | Relacion User -> Order.                                      |
| CONTAINS      | 4772                | Relacion Order -> Product con cantidad.                      |
| RECOMMENDED   | 41                  | Relacion simulada entre usuarios.                            |
| ROUTE_TO      | 10                  | Relacion Location -> User para rutas.                        |

```
docker compose exec neo4j cypher-shell -u neo4j -p test1234 "MATCH (n) RETURN labels(n) AS tipo, count(n) AS cantidad;"
```

```
docker compose exec neo4j cypher-shell -u neo4j -p test1234 "MATCH ()-[r]->() RETURN type(r) AS relacion, count(r) AS cantidad;"
```

## 10. Elasticsearch

El catalogo de productos se indexa en Elasticsearch en el indice productos. La carga se realiza de forma idempotente usando el product\_id como identificador del documento.

| Endpoint                                                      | Uso esperado                               |
|---------------------------------------------------------------|--------------------------------------------|
| GET http://localhost:9200/productos/_count                    | Verifica cantidad de documentos indexados. |
| GET http://localhost:9200/productos/_search?q=name:Producto_1 | Busca productos por nombre.                |

```
curl.exe http://localhost:9200/productos/_count
```

```
curl.exe "http://localhost:9200/productos/_search?q=name:Producto_1"
```

## 11. Metabase y consultas BI

Metabase queda disponible para crear preguntas, consultas SQL y dashboards sobre las vistas OLAP de PostgreSQL. El archivo documentacion/consultas\_metabase.sql contiene consultas base que pueden copiarse en el editor SQL de Metabase.

| Vista/consulta                      | Visualizacion sugerida                          |
|-------------------------------------|-------------------------------------------------|
| v_ingresos_mes_categoria            | Grafico de barras o lineas por mes y categoria. |
| v_horarios_pico                     | Barras de pedidos por hora.                     |
| v_crecimiento_mensual               | Linea de pedidos/ingresos por mes.              |
| v_tendencias_consumo                | Barras por categoria de producto.               |
| v_pedidos_completados_vs_cancelados | Barras apiladas por estado.                     |

## 12. Evidencias de ejecucion

Las siguientes capturas documentan la ejecucion del pipeline y las validaciones realizadas. Se excluye la captura donde se visualizaba la contraseña temporal de Airflow.

```
PS C:\Users\Admin\Downloads\BD_II-proyecto2\proyecto_olap> docker
able_name;"
        table_name
-----
dim_date
dim_location
dim_product
dim_user
fact_reservations
fact_sales
locations
order_details
orders
products
reservations
users
v_actividad_clientes_zona
v_crecimiento_mensual
v_frecuencia_producto_cliente
v_horarios_pico
v_ingresos_mes_categoria
v_pedidos_completados_vs_cancelados
v_tendencias_consumo
(19 rows)

PS C:\Users\Admin\Downloads\BD_II-proyecto2\proyecto_olap> |
```

Evidencia 1. PostgreSQL contiene 19 tablas/vistas entre tablas operacionales, dimensiones, hechos y vistas OLAP.

```
PS C:\Users\Admin\Downloads\BD_II-proyecto2\proyecto_olap> docker compose exec postgres psql -U admin -d restaurantes -c "SELECT COUNT(*) FROM orders;"
count
-----
2000
(1 row)
```

## Evidencia 2. Conteo de la tabla orders con 2000 registros.

```
PS C:\Users\Admin\Downloads\BD_II-proyecto2\proyecto_olap> docker compose exec airflow python /opt/airflow/proyecto_olap/spark_analisis.py
26/06/16 15:39:10 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Conectando a PostgreSQL...
Datos cargados en Spark

=====
ANÁLISIS 1: Producto más popular por mes
=====
+---+-----+-----+
|mes|      name|total_ventas|
+---+-----+-----+
| 1|Producto_19|36|
| 2|Producto_8|28|
| 3|Producto_11|33|
| 4|Producto_5|31|
| 5|Producto_6|33|
| 6|Producto_10|33|
+---+-----+-----+

=====
ANÁLISIS 2: Horarios pico
=====
+---+-----+
|hora|cantidad_pedidos|
+---+-----+
| 22|97|
| 23|91|
| 14|91|
| 13|90|
| 2|89|
| 18|89|
| 3|88|
| 17|88|
| 9|87|
| 5|86|
| 21|86|
| 19|85|
| 0|85|
| 16|84|
| 15|83|
| 1|82|
| 10|80|
| 12|77|
| 7|77|
| 4|76|
| 11|76|
| 6|74|
| 20|72|
```

## Evidencia 3. Spark ejecuta analisis de producto mas popular por mes y horarios pico.

```

| 8| 67|
+---+-----+

=====
ANÁLISIS 3: Crecimiento mensual
=====
26/06/16 15:39:19 WARN WindowExec: No Partition Defined for Win
26/06/16 15:39:19 WARN WindowExec: No Partition Defined for Win
26/06/16 15:39:19 WARN WindowExec: No Partition Defined for Win
26/06/16 15:39:19 WARN WindowExec: No Partition Defined for Win
26/06/16 15:39:19 WARN WindowExec: No Partition Defined for Win
26/06/16 15:39:19 WARN WindowExec: No Partition Defined for Win
26/06/16 15:39:19 WARN WindowExec: No Partition Defined for Win
26/06/16 15:39:19 WARN WindowExec: No Partition Defined for Win
26/06/16 15:39:19 WARN WindowExec: No Partition Defined for Win
+---+-----+
|mes|total_pedidos|mes_anterior|porcentaje_crecimiento|
+---+-----+
| 1| 331| 0| 0.0|
| 2| 325| 331| -1.812688821752266|
| 3| 353| 325| 8.615384615384615|
| 4| 306| 353| -13.314447592067987|
| 5| 355| 306| 16.013071895424837|
| 6| 330| 355| -7.042253521126761|
+---+-----+

Guardando resultados...
26/06/16 15:39:22 WARN WindowExec: No Partition Defined for Win
26/06/16 15:39:22 WARN WindowExec: No Partition Defined for Win
26/06/16 15:39:22 WARN WindowExec: No Partition Defined for Win
26/06/16 15:39:22 WARN WindowExec: No Partition Defined for Win
26/06/16 15:39:22 WARN WindowExec: No Partition Defined for Win
26/06/16 15:39:22 WARN WindowExec: No Partition Defined for Win
26/06/16 15:39:22 WARN WindowExec: No Partition Defined for Win
26/06/16 15:39:22 WARN WindowExec: No Partition Defined for Win
26/06/16 15:39:22 WARN WindowExec: No Partition Defined for Win
Resultados guardados en carpeta 'resultados_spark'

Análisis completado exitosamente

```

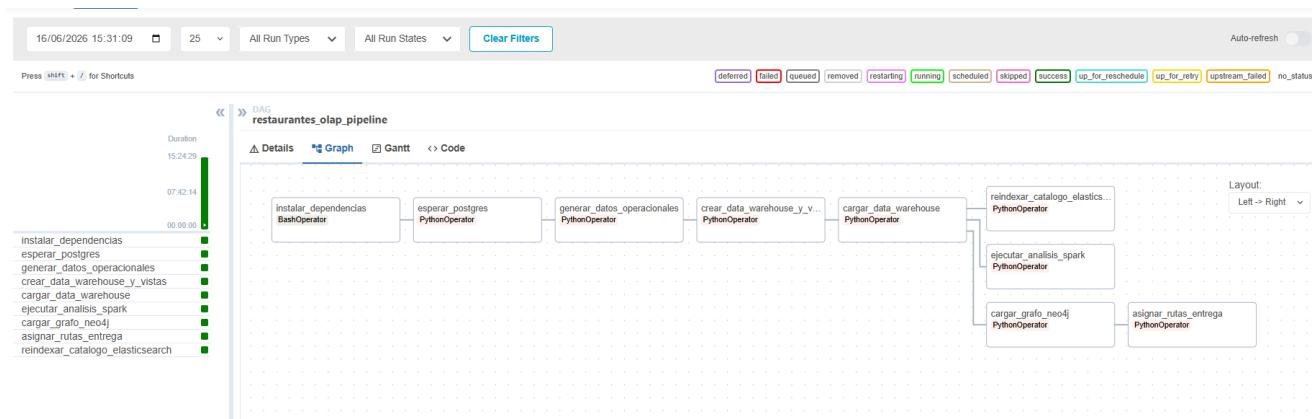
Evidencia 4. Spark calcula crecimiento mensual y finaliza exitosamente.

```

PS C:\Users\Admin\Downloads\BD_II-proyecto2\proyecto_olap> curl.exe http://localhost:9200/productos/_count
{"count":30,"_shards":{"total":1,"successful":1,"skipped":0,"failed":0}}

```

Evidencia 5. Elasticsearch reporta 30 documentos en el índice productos.



Evidencia 6. Airflow muestra el DAG en vista Graph con las tareas en verde.

```
PS C:\Users\Admin\Downloads\BD_II-proyecto2\proyecto_olap> docker compose exec neo4j cypher-shell -u neo4j -p test1234 "MATCH (n) RETURN labels(n) AS tipo, count(n) AS cantidad;"
```

| tipo         | cantidad |
|--------------|----------|
| ["User"]     | 100      |
| ["Product"]  | 30       |
| ["Order"]    | 2000     |
| ["Location"] | 1        |

4 rows  
ready to start consuming query after 61 ms, results consumed after another 6 ms

Evidencia 7. Neo4j contiene nodos User, Product, Order y Location.

```
PS C:\Users\Admin\Downloads\BD_II-proyecto2\proyecto_olap> docker compose exec neo4j cypher-shell -u neo4j -p test1234 "MATCH ()-[r]->() RETURN type(r) AS relacion, count(r) AS cantidad;"
```

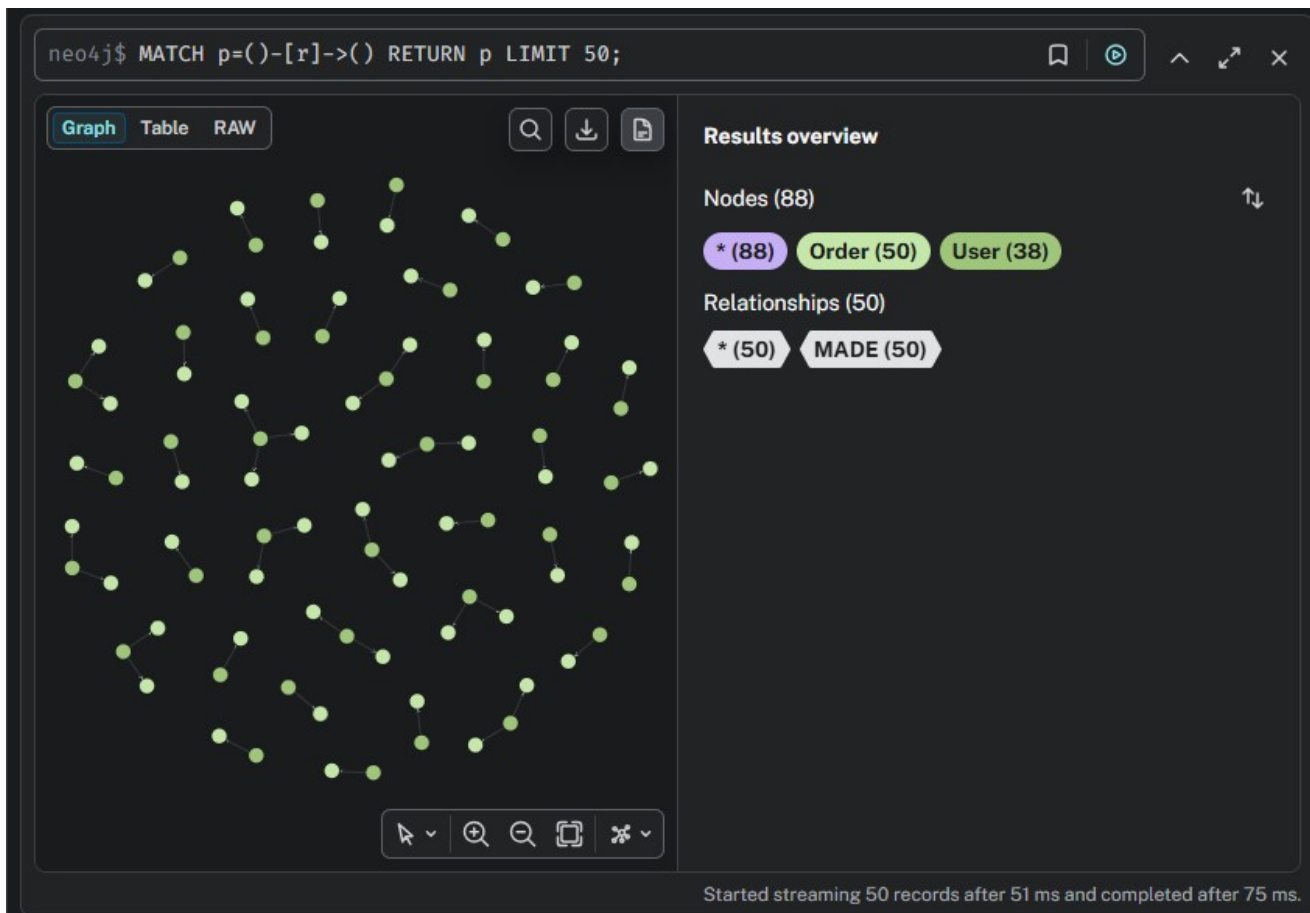
| relacion      | cantidad |
|---------------|----------|
| "MADE"        | 2000     |
| "CONTAINS"    | 4772     |
| "RECOMMENDED" | 41       |
| "ROUTE_TO"    | 10       |

4 rows  
ready to start consuming query after 75 ms, results consumed after another 26 ms

Evidencia 8. Neo4j contiene relaciones MADE, CONTAINS, RECOMMENDED y ROUTE\_TO.

```
PS C:\Users\Admin\Downloads\BD_II-proyecto2\proyecto_olap> docker compose exec airflow ls /opt/airflow/proyecto_olap/resultados_spark
crecimiento_mensual horarios_pico top_producto_mes
```

Evidencia 9. Carpeta resultados\_spark con salidas crecimiento\_mensual, horarios\_pico y top\_producto\_mes.



Evidencia 10. Visualizacion del grafo en Neo4j Browser.

```
p5 C:\Users\Admin\Downloads\BD-II-proyecto2\proyecto_olap> curl.exe "http://localhost:9200/productos/_search?q=name:Producto_1"
{"took":8,"timed_out":false,"_shards":{"total":1,"successful":1,"skipped":0,"failed":0},"hits":{"total":{"value":1,"relation":"eq"},"max_score":3.028522,"hits":[{"_index":"productos","_id":"2","_score":3.028522,"_source":{"product_id": 2, "name": "Producto_1", "category": "Postre", "price": 16.02}}]}}
```

Evidencia 11. Busqueda en Elasticsearch para Producto\_1 con resultado encontrado.

## 13. Checklist de entrega

| Elemento                       | Estado                                                               |
|--------------------------------|----------------------------------------------------------------------|
| docker-compose.yml actualizado | Confirmado. Airflow construye imagen personalizada.                  |
| docker/airflow/Dockerfile      | Confirmado. Instala Java, PySpark y librerías Python.                |
| driver JDBC en drivers/        | Confirmado: postgresql-42.7.3.jar.                                   |
| DAG en airflow/dags/           | Confirmado: restaurantes_olap_dag.py detectado por Airflow.          |
| Capturas sin contraseñas       | Pendiente solo de renombrar/eliminar captura sensible si aun existe. |
| README.md                      | Recomendado en proyecto_olap/README.md.                              |
| Pipeline probado               | Confirmado con DAG exitoso y validaciones por consola.               |

## 14. Problemas encontrados y solucionados

| Problema                                           | Causa                                                                                      | Solucion aplicada                                                         |
|----------------------------------------------------|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| Airflow no detectaba PySpark                       | El servicio usaba imagen oficial apache/airflow sin construir el Dockerfile personalizado. | Se configuro build en el servicio airflow y se reconstruyo la imagen.     |
| docker compose build indicaba No services to build | Docker Compose estaba leyendo un archivo sin seccion build o una copia                     | Se reemplazo el compose local por el actualizado y se ejecuto build sobre |

|                                |                                                       |                                                                                                    |
|--------------------------------|-------------------------------------------------------|----------------------------------------------------------------------------------------------------|
|                                | local desactualizada.                                 | airflow.                                                                                           |
| ModuleNotFoundError: pyspark   | Airflow seguía usando la imagen anterior.             | Se verificó con docker inspect y se recreó el contenedor con proyecto_olap_airflow_pyspark:latest. |
| relation orders does not exist | PostgreSQL estaba vacío antes de ejecutar el DAG.     | Se ejecutó el DAG para generar datos y crear/cargar el DW.                                         |
| Warnings WindowExec en Spark   | Uso de ventana global sin partición para lag mensual. | No afecta el resultado con volumen pequeño de datos; queda como advertencia de rendimiento.        |

## 15. Guía rápida de reproducción

1. Clonar o descargar el repositorio y ubicarse en la carpeta proyecto\_olap.
2. Ejecutar docker compose up -d --build.
3. Verificar contenedores con docker compose ps.
4. Abrir Airflow en <http://localhost:8080> o ejecutar el DAG por consola.
5. Validar PostgreSQL con el conteo de tablas y `SELECT COUNT(*) FROM orders`.
6. Ejecutar o revisar los resultados de Spark.
7. Validar Elasticsearch con `/productos/_count` y una búsqueda por nombre.
8. Validar Neo4j con conteo de nodos y relaciones.
9. Abrir Metabase en <http://localhost:3000> para crear dashboards con las vistas OLAP.

## 16. Referencias técnicas

- **Docker Compose documentation:** <https://docs.docker.com/compose/>
- **Docker Compose file reference:** <https://docs.docker.com/reference/compose-file/>
- **Apache Airflow - DAGs:** <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dags.html>
- **Apache Spark SQL JDBC data source:** <https://spark.apache.org/docs/latest/sql-data-sources-jdbc.html>
- **PostgreSQL information\_schema:** <https://www.postgresql.org/docs/current/information-schema.html>
- **Neo4j Cypher MATCH:** <https://neo4j.com/docs/cypher-manual/current/clauses/match/>
- **Elasticsearch Count API:** <https://www.elastic.co/docs/api/doc/elasticsearch/operation/operation-count>
- **Metabase documentation:** <https://www.metabase.com/docs/latest/>

- Kimball Group - Star Schema and OLAP Cube: <https://www.kimballgroup.com/data-warehouse-business-intelligence-resources/kimball-techniques/dimensional-modeling-techniques/star-schema-olap-cube/>
- PostgreSQL - CREATE VIEW: <https://www.postgresql.org/docs/current/sql-createview.html>

## 17. Referencia detallada de endpoints y puntos de integracion

Esta seccion amplia la documentacion de endpoints con un formato similar al de documentaciones publicas de APIs: metodo, URL, parametros, autenticacion, ejemplos, respuestas y errores. En este proyecto no existe una API REST propia desarrollada por el equipo; los endpoints documentados pertenecen a los servicios levantados por Docker Compose y a los puntos de integracion usados por Airflow, PostgreSQL, Spark, Neo4j, Elasticsearch y Metabase.

### 17.1 Criterio de documentacion usado

Para evitar una lista superficial de puertos, cada punto de consumo se documenta como si fuera una referencia tecnica. El formato incluye: nombre del endpoint, metodo o protocolo, ruta, parametros de ruta, parametros de consulta, cuerpo de solicitud, headers, respuesta exitosa, errores comunes, ejemplo ejecutable y observaciones de uso dentro del pipeline.

| Elemento documentado | Razon                                                                                             |
|----------------------|---------------------------------------------------------------------------------------------------|
| Metodo / protocolo   | Permite distinguir entre HTTP REST, Bolt, JDBC, SQL por psql o interfaz web.                      |
| URL o DSN            | Permite reproducir la conexion desde navegador, curl, scripts o contenedores.                     |
| Parametros           | Aclara que campos son obligatorios, opcionales y que valores son validos.                         |
| Cuerpo de solicitud  | Explica los JSON o consultas SQL/Cypher enviados.                                                 |
| Respuesta esperada   | Define como se sabe que el endpoint funciona correctamente.                                       |
| Errores comunes      | Ayuda a diagnosticar fallos de autenticacion, servicio caido, indice inexistente o tablas vacias. |
| Ejemplo              | Da un comando listo para ejecutar y capturar evidencia.                                           |

*El estilo se inspira en documentaciones como Spotify Web API, OpenAPI y referencias oficiales de Airflow y Elasticsearch, adaptado a un proyecto academico de datos.*

### 17.2 Resumen de endpoints por servicio

| Servicio             | Tipo           | Endpoint principal                                                      | Uso en el proyecto                                                  | Criticidad |
|----------------------|----------------|-------------------------------------------------------------------------|---------------------------------------------------------------------|------------|
| Airflow UI           | HTTP Web UI    | <a href="http://localhost:8080">http://localhost:8080</a>               | Monitorear y ejecutar DAG restaurantes_olap_pipeline.               | Alta       |
| Airflow REST API     | HTTP JSON API  | <a href="http://localhost:8080/api/v1">http://localhost:8080/api/v1</a> | Consultar DAGs, ejecuciones y task instances de forma programatica. | Media      |
| PostgreSQL           | TCP/JDBC/psql  | localhost:5432 / postgres:5432                                          | Base operacional y Data Warehouse OLAP.                             | Alta       |
| Spark Master UI      | HTTP Web UI    | <a href="http://localhost:8081">http://localhost:8081</a>               | Ver disponibilidad del master Spark.                                | Media      |
| Spark Master interno | Spark protocol | spark://spark-master:7077                                               | Conexion worker-master dentro de Docker.                            | Media      |
| Neo4j Browser        | HTTP Web UI    | <a href="http://localhost:7474">http://localhost:7474</a>               | Explorar grafo visualmente.                                         | Media      |
| Neo4j Bolt           | Bolt           | bolt://localhost:7687 / bolt://neo4j:7687                               | Conexion de scripts Python y cypher-shell.                          | Alta       |
| Elasticsearch        | HTTP REST      | <a href="http://localhost:9200">http://localhost:9200</a>               | Contar, buscar e indexar productos.                                 | Alta       |
| Metabase             | HTTP Web UI    | <a href="http://localhost:3000">http://localhost:3000</a>               | Crear consultas y dashboards sobre vistas OLAP.                     | Media      |

### 17.3 Airflow - endpoints de orquestacion

Airflow es el componente que coordina el proceso. El DAG principal se llama restaurantes\_olap\_pipeline. Para la presentacion, la interfaz web es la evidencia visual principal, pero tambien se documentan



endpoints REST de Airflow 2.x para consultar o disparar ejecuciones desde herramientas como curl o Postman.

### 17.3.1 Abrir interfaz web de Airflow

| Campo                | Detalle                                                                                                                          |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Metodo               | GET                                                                                                                              |
| URL base / ruta      | http://localhost:8080                                                                                                            |
| Proposito            | Ingresar al panel de Airflow para revisar el estado del DAG, historial de ejecuciones, vista Graph, logs y tareas en verde/rojo. |
| Autenticacion        | Usuario administrador generado por airflow standalone. No documentar ni subir capturas con la contraseña.                        |
| Headers recomendados | No requiere headers especiales desde navegador.                                                                                  |

Respuesta exitosa:

| Estado | Contenido esperado                                                                               |
|--------|--------------------------------------------------------------------------------------------------|
| 200 OK | Carga la interfaz de Airflow. Debe visualizarse el DAG <code>restaurantes_olap_pipeline</code> . |

Errores esperados:

| Estado             | Causa probable                           | Accion sugerida                                                                       |
|--------------------|------------------------------------------|---------------------------------------------------------------------------------------|
| Connection refused | El contenedor airflow no esta levantado. | Ejecutar <code>docker compose ps</code> y <code>docker compose up -d airflow</code> . |
| 401/Redirect login | Sesion no iniciada.                      | Ingresar con usuario admin y la contraseña local generada.                            |
| 404                | Ruta incorrecta o puerto no expuesto.    | Confirmar que el puerto 8080 este mapeado en <code>docker-compose.yml</code> .        |

Ejemplo de consumo:

```
# Abrir desde navegador
http://localhost:8080
```

```
# Verificar contenedor
docker compose ps airflow
```

*Uso en la entrega: capturar la vista Graph con todas las tareas en verde.*

### 17.3.2 Listar DAGs por REST API

| Campo                | Detalle                                                                                           |
|----------------------|---------------------------------------------------------------------------------------------------|
| Metodo               | GET                                                                                               |
| URL base / ruta      | http://localhost:8080/api/v1/dags                                                                 |
| Proposito            | Consultar los DAGs registrados en Airflow sin usar la interfaz grafica.                           |
| Autenticacion        | Basic Auth con usuario admin y contraseña local. Usar placeholders; no subir credenciales reales. |
| Headers recomendados | Accept: application/json                                                                          |

Parametros de consulta:

| Nombre      | Tipo    | Obligatorio | Descripcion                         | Ejemplo / valor permitido |
|-------------|---------|-------------|-------------------------------------|---------------------------|
| limit       | integer | No          | Cantidad maxima de DAGs a devolver. | 100                       |
| offset      | integer | No          | Desplazamiento para paginacion.     | 0                         |
| only_active | boolean | No          | Filtra DAGs activos.                | true                      |

Respuesta exitosa:

| Estado | Contenido esperado                                                             |
|--------|--------------------------------------------------------------------------------|
| 200 OK | JSON con una lista de DAGs. Debe aparecer dag_id = restaurantes_olap_pipeline. |

Errores esperados:

| Estado | Causa probable                       | Accion sugerida                      |
|--------|--------------------------------------|--------------------------------------|
| 401    | Credenciales invalidas o faltantes.  | Usar -u admin:<AIRFLOW_PASSWORD>.    |
| 403    | Usuario sin permisos suficientes.    | Usar usuario administrador local.    |
| 500    | Airflow webserver con error interno. | Revisar docker compose logs airflow. |

Ejemplo de consumo:

```
curl.exe -u admin:<AIRFLOW_PASSWORD> -H "Accept: application/json" "http://localhost:8080/api/v1/dags?limit=100&offset=0"
```

### 17.3.3 Obtener detalle del DAG del proyecto

| Campo                | Detalle                                                                                                     |
|----------------------|-------------------------------------------------------------------------------------------------------------|
| Metodo               | GET                                                                                                         |
| URL base / ruta      | http://localhost:8080/api/v1/dags/{dag_id}                                                                  |
| Proposito            | Consultar metadatos de un DAG especifico: id, estado pausado/activo, archivo fuente y parametros generales. |
| Autenticacion        | Basic Auth con usuario admin.                                                                               |
| Headers recomendados | Accept: application/json                                                                                    |

Parametros de ruta:

| Nombre | Tipo   | Obligatorio | Descripcion            | Ejemplo                    |
|--------|--------|-------------|------------------------|----------------------------|
| dag_id | string | Si          | Identificador del DAG. | restaurantes_olap_pipeline |

Respuesta exitosa:

| Estado | Contenido esperado                        |
|--------|-------------------------------------------|
| 200 OK | JSON con los detalles del DAG solicitado. |

Errores esperados:

| Estado | Causa probable                                         | Accion sugerida                                                             |
|--------|--------------------------------------------------------|-----------------------------------------------------------------------------|
| 401    | No autenticado.                                        | Enviar Basic Auth.                                                          |
| 404    | El dag_id no existe o Airflow no cargo el archivo DAG. | Ejecutar airflow dags list y revisar airflow/dags/restaurantes_olap_dag.py. |

Ejemplo de consumo:

```
curl.exe -u admin:<AIRFLOW_PASSWORD> -H "Accept: application/json" "http://localhost:8080/api/v1/dags/restaurantes_olap_pipeline"
```

### 17.3.4 Crear una ejecucion del DAG por API

| Campo           | Detalle                                                                                                     |
|-----------------|-------------------------------------------------------------------------------------------------------------|
| Metodo          | POST                                                                                                        |
| URL base / ruta | http://localhost:8080/api/v1/dags/{dag_id}/dagRuns                                                          |
| Proposito       | Disparar una nueva ejecucion del pipeline desde una llamada HTTP, equivalente a activar el DAG desde la UI. |
| Autenticacion   | Basic Auth con usuario admin.                                                                               |

|                      |                                                          |
|----------------------|----------------------------------------------------------|
| Headers recomendados | Content-Type: application/json; Accept: application/json |
|----------------------|----------------------------------------------------------|

Parametros de ruta:

| Nombre | Tipo   | Obligatorio | Descripcion            | Ejemplo                    |
|--------|--------|-------------|------------------------|----------------------------|
| dag_id | string | Si          | Identificador del DAG. | restaurantes_olap_pipeline |

Cuerpo de solicitud:

| Campo        | Tipo             | Obligatorio | Descripcion                                                                   | Ejemplo              |
|--------------|------------------|-------------|-------------------------------------------------------------------------------|----------------------|
| dag_run_id   | string           | No          | Identificador manual de la ejecucion. Si se omite, Airflow genera uno.        | manual_olap_001      |
| logical_date | string/date-time | No          | Fecha logica de la ejecucion en formato ISO 8601.                             | 2026-06-16T00:00:00Z |
| conf         | object           | No          | Objeto JSON para parametros dinamicos. En este proyecto puede enviarse vacio. | {}                   |

Respuesta exitosa:

| Estado  | Contenido esperado                                     |
|---------|--------------------------------------------------------|
| 200/201 | JSON con el dag_run_id, estado inicial y fecha logica. |

Errores esperados:

| Estado  | Causa probable                           | Accion sugerida                              |
|---------|------------------------------------------|----------------------------------------------|
| 400     | JSON mal formado o dag_run_id duplicado. | Cambiar dag_run_id o validar comillas.       |
| 401/403 | Autenticacion/permisos.                  | Usar credenciales validas.                   |
| 404     | DAG inexistente.                         | Verificar nombre restaurantes_olap_pipeline. |
| 422     | Campos con tipo/formato invalido.        | Revisar logical_date y estructura JSON.      |

Ejemplo de consumo:

```
curl.exe -X POST -u admin:<AIRFLOW_PASSWORD> -H "Content-Type: application/json" -H "Accept: application/json"
"http://localhost:8080/api/v1/dags/restaurantes_olap_pipeline/dagRuns" -d '{"dag_run_id":"manual_olap_001","logical_date":"2026-06-16T00:00:00Z","conf":{}}'
```

*Para la entrega se uso principalmente airflow dags test por consola; este endpoint se documenta como alternativa programatica.*

### 17.3.5 Consultar ejecuciones del DAG

| Campo                | Detalle                                                                    |
|----------------------|----------------------------------------------------------------------------|
| Metodo               | GET                                                                        |
| URL base / ruta      | http://localhost:8080/api/v1/dags/{dag_id}/dagRuns                         |
| Proposito            | Listar ejecuciones del pipeline para comprobar historial, estado y fechas. |
| Autenticacion        | Basic Auth con usuario admin.                                              |
| Headers recomendados | Accept: application/json                                                   |

Parametros de ruta:

| Nombre | Tipo   | Obligatorio | Descripcion            | Ejemplo                    |
|--------|--------|-------------|------------------------|----------------------------|
| dag_id | string | Si          | Identificador del DAG. | restaurantes_olap_pipeline |

## Parametros de consulta:

| Nombre   | Tipo    | Obligatorio | Descripcion                               | Ejemplo / valor permitido |
|----------|---------|-------------|-------------------------------------------|---------------------------|
| limit    | integer | No          | Cantidad maxima de ejecuciones devueltas. | 25                        |
| offset   | integer | No          | Desplazamiento de paginacion.             | 0                         |
| state    | string  | No          | Filtra por estado.                        | success, failed, running  |
| order_by | string  | No          | Ordenamiento.                             | -start_date               |

## Respuesta exitosa:

| Estado | Contenido esperado                                                                          |
|--------|---------------------------------------------------------------------------------------------|
| 200 OK | JSON con dag_runs. En la evidencia debe existir al menos una ejecucion con state = success. |

## Errores esperados:

| Estado  | Causa probable                | Accion sugerida   |
|---------|-------------------------------|-------------------|
| 401/403 | No autenticado o sin permiso. | Usar Basic Auth.  |
| 404     | DAG inexistente.              | Verificar dag_id. |

## Ejemplo de consumo:

```
curl.exe -u admin:<AIRFLOW_PASSWORD> -H "Accept: application/json"  
"http://localhost:8080/api/v1/dags/restaurantes_olap_pipeline/dagRuns?limit=25&order_by=-start_date"
```

### 17.3.6 Consultar tareas de una ejecucion

| Campo                | Detalle                                                                                                                                   |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Metodo               | GET                                                                                                                                       |
| URL base / ruta      | http://localhost:8080/api/v1/dags/{dag_id}/dagRuns/{dag_run_id}/taskInstances                                                             |
| Proposito            | Consultar el estado individual de cada tarea del pipeline: instalar dependencias, generar datos, cargar DW, Spark, Neo4j y Elasticsearch. |
| Autenticacion        | Basic Auth con usuario admin.                                                                                                             |
| Headers recomendados | Accept: application/json                                                                                                                  |

## Parametros de ruta:

| Nombre     | Tipo   | Obligatorio | Descripcion                    | Ejemplo                          |
|------------|--------|-------------|--------------------------------|----------------------------------|
| dag_id     | string | Si          | Identificador del DAG.         | restaurantes_olap_pipeline       |
| dag_run_id | string | Si          | Identificador de la ejecucion. | manual_2026-06-16T00:00:00+00:00 |

## Respuesta exitosa:

| Estado | Contenido esperado                                                                               |
|--------|--------------------------------------------------------------------------------------------------|
| 200 OK | JSON con task_instances y state por tarea. Todas deben estar en success para una corrida valida. |

## Errores esperados:

| Estado  | Causa probable                | Accion sugerida                                               |
|---------|-------------------------------|---------------------------------------------------------------|
| 404     | DAG run no encontrado.        | Copiar dag_run_id desde la UI o desde el endpoint de dagRuns. |
| 401/403 | No autenticado o sin permiso. | Usar usuario admin.                                           |

Ejemplo de consumo:

```
curl.exe -u admin:<AIRFLOW_PASSWORD> -H "Accept: application/json"  
"http://localhost:8080/api/v1/dags/restaurantes_olap_pipeline/dagRuns/<DAG_RUN_ID>/taskInstances"
```

## 17.4 Elasticsearch - API REST del indice productos

Elasticsearch es el servicio del proyecto que mas se parece a una API REST tradicional. El indice productos contiene el catalogo cargado desde PostgreSQL. En docker-compose.yml se desactiva la seguridad de Elasticsearch para ambiente local academico, por lo que las pruebas con curl no requieren token.

### 17.4.1 Health check del servicio Elasticsearch

| Campo                | Detalle                                                                          |
|----------------------|----------------------------------------------------------------------------------|
| Metodo               | GET                                                                              |
| URL base / ruta      | http://localhost:9200/                                                           |
| Proposito            | Verificar que Elasticsearch responde antes de ejecutar indexaciones o busquedas. |
| Autenticacion        | Sin autenticacion en ambiente local: xpack.security.enabled=false.               |
| Headers recomendados | Accept: application/json                                                         |

Respuesta exitosa:

| Estado | Contenido esperado                                           |
|--------|--------------------------------------------------------------|
| 200 OK | JSON con informacion del cluster, version y nombre del nodo. |

Errores esperados:

| Estado             | Causa probable                              | Accion sugerida                           |
|--------------------|---------------------------------------------|-------------------------------------------|
| Connection refused | El contenedor elasticsearch no esta activo. | Ejecutar docker compose ps elasticsearch. |
| 503                | Nodo iniciando o cluster no disponible.     | Esperar unos segundos y revisar logs.     |

Ejemplo de consumo:

```
curl.exe http://localhost:9200/
```

### 17.4.2 Contar documentos del indice productos

| Campo                | Detalle                                                                                     |
|----------------------|---------------------------------------------------------------------------------------------|
| Metodo               | GET                                                                                         |
| URL base / ruta      | http://localhost:9200/productos/_count                                                      |
| Proposito            | Validar que el catalogo fue indexado. En las evidencias el conteo esperado es 30 productos. |
| Autenticacion        | Sin autenticacion en ambiente local.                                                        |
| Headers recomendados | Accept: application/json                                                                    |

Parametros de consulta:

| Nombre | Tipo   | Obligatorio | Descripcion                                                | Ejemplo / valor permitido |
|--------|--------|-------------|------------------------------------------------------------|---------------------------|
| q      | string | No          | Consulta simple para contar solo documentos que coincidan. | name:Producto_1           |

Cuerpo de solicitud:

| Campo | Tipo   | Obligatorio | Descripcion                                                         | Ejemplo          |
|-------|--------|-------------|---------------------------------------------------------------------|------------------|
| query | object | No          | Consulta Query DSL para filtrar conteo. Si se omite, usa match_all. | {"match_all":{}} |

Respuesta exitosa:

| Estado | Contenido esperado                                      |
|--------|---------------------------------------------------------|
| 200 OK | JSON con count. Resultado verificado: {"count":30,...}. |

Errores esperados:

| Estado | Causa probable              | Accion sugerida                                        |
|--------|-----------------------------|--------------------------------------------------------|
| 404    | Indice productos no existe. | Ejecutar DAG o tarea reindexar_catalogo_elasticsearch. |
| 400    | Query string invalida.      | Revisar sintaxis q=campo:valor.                        |

Ejemplo de consumo:

# Conteo total

curl.exe http://localhost:9200/productos/\_count

# Conteo con filtro simple

curl.exe "http://localhost:9200/productos/\_count?q=name:Producto\_1"

### 17.4.3 Buscar productos por query string

| Campo                | Detalle                                                                                                                    |
|----------------------|----------------------------------------------------------------------------------------------------------------------------|
| Metodo               | GET                                                                                                                        |
| URL base / ruta      | http://localhost:9200/productos/_search                                                                                    |
| Proposito            | Buscar productos del catalogo por campos como name o category. Es el endpoint usado para demostrar busqueda tipo catalogo. |
| Autenticacion        | Sin autenticacion en ambiente local.                                                                                       |
| Headers recomendados | Accept: application/json                                                                                                   |

Parametros de consulta:

| Nombre | Tipo    | Obligatorio | Descripcion                               | Ejemplo / valor permitido |
|--------|---------|-------------|-------------------------------------------|---------------------------|
| q      | string  | Si          | Consulta simple de busqueda.              | name:Producto_1           |
| size   | integer | No          | Cantidad maxima de documentos a devolver. | 10                        |
| from   | integer | No          | Desplazamiento para paginacion.           | 0                         |
| sort   | string  | No          | Ordenamiento por campo.                   | price:asc                 |

Respuesta exitosa:

| Estado | Contenido esperado                                                                           |
|--------|----------------------------------------------------------------------------------------------|
| 200 OK | JSON con hits.total.value y arreglo hits.hits. En evidencia Producto_1 devuelve 1 resultado. |

Errores esperados:

| Estado | Causa probable          | Accion sugerida                        |
|--------|-------------------------|----------------------------------------|
| 404    | Indice no existe.       | Ejecutar reindexacion.                 |
| 400    | Consulta q mal formada. | Validar campo y caracteres especiales. |

Ejemplo de consumo:

```
curl.exe "http://localhost:9200/productos/_search?q=name:Producto_1&size=10&from=0"
```

#### 17.4.4 Buscar productos con Query DSL

| Campo                | Detalle                                                                                                                 |
|----------------------|-------------------------------------------------------------------------------------------------------------------------|
| Metodo               | POST                                                                                                                    |
| URL base / ruta      | http://localhost:9200/productos/_search                                                                                 |
| Proposito            | Ejecutar una busqueda mas estructurada usando JSON. Es la forma recomendada cuando hay filtros o condiciones multiples. |
| Autenticacion        | Sin autenticacion en ambiente local.                                                                                    |
| Headers recomendados | Content-Type: application/json; Accept: application/json                                                                |

Cuerpo de solicitud:

| Campo | Tipo    | Obligatorio | Descripcion                                 | Ejemplo                           |
|-------|---------|-------------|---------------------------------------------|-----------------------------------|
| query | object  | Si          | Objeto de consulta Elasticsearch Query DSL. | {"match": {"name": "Producto_1"}} |
| size  | integer | No          | Cantidad maxima de resultados.              | 10                                |
| from  | integer | No          | Offset de resultados.                       | 0                                 |

Respuesta exitosa:

| Estado | Contenido esperado                                                 |
|--------|--------------------------------------------------------------------|
| 200 OK | JSON con resultados dentro de hits.hits y metadatos de puntuacion. |

Errores esperados:

| Estado | Causa probable                         | Accion sugerida                |
|--------|----------------------------------------|--------------------------------|
| 400    | JSON invalido o Query DSL mal formada. | Validar comillas y estructura. |
| 404    | Indice inexistente.                    | Reindexar catalogo.            |

Ejemplo de consumo:

```
curl.exe -X POST "http://localhost:9200/productos/_search" -H "Content-Type: application/json" -d '{"query":{"match":{"name":"Producto_1"},"size":10}'
```

#### 17.4.5 Consultar documento por ID de producto

| Campo                | Detalle                                                                         |
|----------------------|---------------------------------------------------------------------------------|
| Metodo               | GET                                                                             |
| URL base / ruta      | http://localhost:9200/productos/_doc/{product_id}                               |
| Proposito            | Recuperar un producto especifico indexado usando el identificador del producto. |
| Autenticacion        | Sin autenticacion en ambiente local.                                            |
| Headers recomendados | Accept: application/json                                                        |

Parametros de ruta:

| Nombre     | Tipo           | Obligatorio | Descripcion                                      | Ejemplo |
|------------|----------------|-------------|--------------------------------------------------|---------|
| product_id | integer/string | Si          | ID del producto usado como _id en Elasticsearch. | 1       |

Respuesta exitosa:

| Estado | Contenido esperado                                    |
|--------|-------------------------------------------------------|
| 200 OK | JSON con _source: product_id, name, category y price. |

Errores esperados:

| Estado | Causa probable                     | Accion sugerida                      |
|--------|------------------------------------|--------------------------------------|
| 404    | Producto no existe o indice vacio. | Verificar count y product_id valido. |

Ejemplo de consumo:

curl.exe http://localhost:9200/productos/\_doc/1

### 17.4.6 Indexar o reemplazar un producto

| Campo                | Detalle                                                                                                              |
|----------------------|----------------------------------------------------------------------------------------------------------------------|
| Metodo               | PUT                                                                                                                  |
| URL base / ruta      | http://localhost:9200/productos/_doc/{product_id}                                                                    |
| Proposito            | Endpoint usado conceptualmente por cargar_elasticsearch.py para insertar/actualizar documentos de forma idempotente. |
| Autenticacion        | Sin autenticacion en ambiente local.                                                                                 |
| Headers recomendados | Content-Type: application/json; Accept: application/json                                                             |

Parametros de ruta:

| Nombre     | Tipo           | Obligatorio | Descripcion                       | Ejemplo |
|------------|----------------|-------------|-----------------------------------|---------|
| product_id | integer/string | Si          | Identificador unico del producto. | 1       |

Cuerpo de solicitud:

| Campo      | Tipo    | Obligatorio | Descripcion                 | Ejemplo    |
|------------|---------|-------------|-----------------------------|------------|
| product_id | integer | Si          | Identificador del producto. | 1          |
| name       | string  | Si          | Nombre del producto.        | Producto_1 |
| category   | string  | Si          | Categoria del producto.     | Postre     |
| price      | number  | Si          | Precio del producto.        | 16.02      |

Respuesta exitosa:

| Estado  | Contenido esperado                   |
|---------|--------------------------------------|
| 200/201 | JSON con result = updated o created. |

Errores esperados:

| Estado | Causa probable                   | Accion sugerida                   |
|--------|----------------------------------|-----------------------------------|
| 400    | JSON invalido o tipo incorrecto. | Validar estructura del documento. |
| 503    | Elasticsearch no disponible.     | Esperar salud del cluster.        |

Ejemplo de consumo:

```
curl.exe -X PUT "http://localhost:9200/productos/_doc/1" -H "Content-Type: application/json" -d
{"product_id":1,"name":"Producto_1","category":"Postre","price":16.02}"
```



## 17.5 PostgreSQL - conexion y consultas de validacion

PostgreSQL no expone endpoints HTTP en este proyecto. Su punto de integracion es TCP/JDBC/psql. Para efectos de documentacion, se documentan las consultas de validacion como operaciones reproducibles, porque son equivalentes a endpoints de lectura para comprobar el estado del sistema.

### 17.5.1 Conexion psql a base restaurantes

| Campo                | Detalle                                                                                                     |
|----------------------|-------------------------------------------------------------------------------------------------------------|
| Metodo               | psql sobre TCP dentro de Docker                                                                             |
| URL base / ruta      | postgresql://admin:<PASSWORD>@localhost:5432/restaurantes o host interno postgres:5432                      |
| Proposito            | Abrir una sesion SQL o ejecutar comandos contra la base restaurantes.                                       |
| Autenticacion        | Usuario admin y contrasena configurada en docker-compose.yml. No exponer credenciales en capturas publicas. |
| Headers recomendados | No aplica.                                                                                                  |

Cuerpo de solicitud:

| Campo | Tipo   | Obligatorio | Descripcion                                            | Ejemplo                      |
|-------|--------|-------------|--------------------------------------------------------|------------------------------|
| SQL   | string | Si          | Consulta SQL enviada con -c o desde shell interactivo. | SELECT COUNT(*) FROM orders; |

Respuesta exitosa:

| Estado      | Contenido esperado                |
|-------------|-----------------------------------|
| 0 exit code | psql imprime resultados en tabla. |

Errores esperados:

| Estado                         | Causa probable               | Accion sugerida                                     |
|--------------------------------|------------------------------|-----------------------------------------------------|
| connection refused             | PostgreSQL no esta activo.   | docker compose ps postgres.                         |
| password authentication failed | Credenciales incorrectas.    | Revisar docker-compose.yml.                         |
| database does not exist        | Base restaurantes no creada. | Recrear contenedor o revisar variables POSTGRES_DB. |

Ejemplo de consumo:

```
docker compose exec postgres psql -U admin -d restaurantes -c "SELECT version();"
```

### 17.5.2 Listar tablas y vistas del esquema public

| Campo                | Detalle                                                                            |
|----------------------|------------------------------------------------------------------------------------|
| Metodo               | SQL SELECT                                                                         |
| URL base / ruta      | Base restaurantes / information_schema.tables                                      |
| Proposito            | Verificar que el DAG creo tablas operacionales, dimensiones, hechos y vistas OLAP. |
| Autenticacion        | Conexion psql con usuario admin.                                                   |
| Headers recomendados | No aplica.                                                                         |

Cuerpo de solicitud:

| Campo        | Tipo   | Obligatorio | Descripcion                    | Ejemplo |
|--------------|--------|-------------|--------------------------------|---------|
| table_schema | string | Si          | Filtro del esquema consultado. | public  |

Respuesta exitosa:

| Estado        | Contenido esperado                                                                   |
|---------------|--------------------------------------------------------------------------------------|
| Resultado SQL | Debe listar 19 filas incluyendo dim_date, fact_sales, orders, products y vistas v_*. |

Errores esperados:

| Estado                  | Causa probable                                      | Accion sugerida                                                   |
|-------------------------|-----------------------------------------------------|-------------------------------------------------------------------|
| 0 rows                  | El DAG no se ha ejecutado o el volumen esta limpio. | Ejecutar airflow dags test restaurantes_olap_pipeline 2026-06-16. |
| relation does not exist | Consulta mal escrita o esquema inexistente.         | Revisar SQL.                                                      |

Ejemplo de consumo:

```
docker compose exec postgres psql -U admin -d restaurantes -c "SELECT table_name FROM information_schema.tables WHERE table_schema='public' ORDER BY table_name;"
```

### 17.5.3 Conteo de ordenes cargadas

| Campo                | Detalle                                                                                |
|----------------------|----------------------------------------------------------------------------------------|
| Metodo               | SQL SELECT                                                                             |
| URL base / ruta      | Base restaurantes / tabla orders                                                       |
| Proposito            | Validar el volumen de datos operacionales generado. La evidencia muestra 2000 ordenes. |
| Autenticacion        | Conexion psql con usuario admin.                                                       |
| Headers recomendados | No aplica.                                                                             |

Respuesta exitosa:

| Estado        | Contenido esperado |
|---------------|--------------------|
| Resultado SQL | count = 2000.      |

Errores esperados:

| Estado                           | Causa probable                        | Accion sugerida                            |
|----------------------------------|---------------------------------------|--------------------------------------------|
| relation "orders" does not exist | No se ejecuto la generacion de datos. | Ejecutar DAG completo.                     |
| count = 0                        | Datos no cargados o fueron limpiados. | Revisar tarea generar_datos_operacionales. |

Ejemplo de consumo:

```
docker compose exec postgres psql -U admin -d restaurantes -c "SELECT COUNT(*) FROM orders;"
```

### 17.5.4 Consultar vista OLAP de horarios pico

| Campo                | Detalle                                                                                                         |
|----------------------|-----------------------------------------------------------------------------------------------------------------|
| Metodo               | SQL SELECT                                                                                                      |
| URL base / ruta      | Base restaurantes / vista v_horarios_pico                                                                       |
| Proposito            | Obtener las horas con mayor cantidad de pedidos para alimentar visualizaciones o validar resultados analiticos. |
| Autenticacion        | Conexion psql con usuario admin.                                                                                |
| Headers recomendados | No aplica.                                                                                                      |

Cuerpo de solicitud:

| Campo | Tipo    | Obligatorio | Descripcion               | Ejemplo |
|-------|---------|-------------|---------------------------|---------|
| LIMIT | integer | No          | Cantidad maxima de filas. | 10      |

Respuesta exitosa:

| Estado        | Contenido esperado                 |
|---------------|------------------------------------|
| Resultado SQL | Filas con hora y cantidad pedidos. |

Errores esperados:

| Estado                  | Causa probable                  | Accion sugerida                               |
|-------------------------|---------------------------------|-----------------------------------------------|
| relation does not exist | dw_schema.sql no fue ejecutado. | Ejecutar tarea crear_data_warehouse_y_vistas. |

Ejemplo de consumo:

```
docker compose exec postgres psql -U admin -d restaurantes -c "SELECT * FROM v_horarios_pico LIMIT 10;"
```

### 17.5.5 Conexion JDBC usada por Spark

| Campo                | Detalle                                                                                                              |
|----------------------|----------------------------------------------------------------------------------------------------------------------|
| Metodo               | JDBC                                                                                                                 |
| URL base / ruta      | jdbc:postgresql://postgres:5432/restaurantes                                                                         |
| Proposito            | Permitir que spark_analisis.py lea orders, order_details, products y users desde PostgreSQL dentro de la red Docker. |
| Autenticacion        | Usuario admin y contrasena admin123 configurados como properties del lector JDBC.                                    |
| Headers recomendados | No aplica.                                                                                                           |

Cuerpo de solicitud:

| Campo      | Tipo   | Obligatorio | Descripcion                           | Ejemplo                                                  |
|------------|--------|-------------|---------------------------------------|----------------------------------------------------------|
| driver     | string | Si          | Clase del driver JDBC.                | org.postgresql.Driver                                    |
| spark.jars | path   | Si          | Ruta al driver postgresql-42.7.3.jar. | /opt/airflow/proyecto_olap/drivers/postgresql-42.7.3.jar |
| table      | string | Si          | Tabla que Spark lee mediante JDBC.    | orders                                                   |

Respuesta exitosa:

| Estado                  | Contenido esperado                                           |
|-------------------------|--------------------------------------------------------------|
| Spark DataFrame cargado | El script imprime Datos cargados en Spark y genera analisis. |

Errores esperados:

| Estado                  | Causa probable                              | Accion sugerida                                           |
|-------------------------|---------------------------------------------|-----------------------------------------------------------|
| ClassNotFoundException  | Driver JDBC no encontrado.                  | Confirmar carpeta drivers y ruta spark.jars.              |
| relation does not exist | Tabla no creada.                            | Ejecutar DAG antes de Spark.                              |
| Connection refused      | Postgres no disponible desde la red Docker. | Usar host postgres, no localhost, dentro de contenedores. |

Ejemplo de consumo:

```
docker compose exec airflow python /opt/airflow/proyecto_olap/spark_analisis.py
```

## 17.6 Neo4j - endpoints de grafo y consultas Cypher

Neo4j se consume por dos vías: interfaz Browser para visualización y protocolo Bolt para scripts Python/cypher-shell. Las consultas Cypher funcionan como operaciones de lectura para validar nodos, relaciones y rutas.

### 17.6.1 Abrir Neo4j Browser

| Campo                | Detalle                                                                                            |
|----------------------|----------------------------------------------------------------------------------------------------|
| Metodo               | GET                                                                                                |
| URL base / ruta      | http://localhost:7474                                                                              |
| Proposito            | Visualizar el grafo, ejecutar consultas Cypher y capturar una representación visual del resultado. |
| Autenticacion        | Usuario neo4j y contraseña configurada en docker-compose.yml.                                      |
| Headers recomendados | No requiere headers desde navegador.                                                               |

Respuesta exitosa:

| Estado | Contenido esperado                  |
|--------|-------------------------------------|
| 200 OK | Carga la interfaz de Neo4j Browser. |

Errores esperados:

| Estado             | Causa probable              | Acción sugerida          |
|--------------------|-----------------------------|--------------------------|
| Connection refused | Contenedor neo4j no activo. | docker compose ps neo4j. |
| Auth failed        | Credenciales incorrectas.   | Revisar NEO4J_AUTH.      |

Ejemplo de consumo:

```
# Abrir navegador  
http://localhost:7474
```

```
# Consulta visual sugerida  
MATCH p=()-[r]->() RETURN p LIMIT 50;
```

### 17.6.2 Conexión Bolt para scripts y cypher-shell

| Campo                | Detalle                                                                                   |
|----------------------|-------------------------------------------------------------------------------------------|
| Metodo               | Bolt                                                                                      |
| URL base / ruta      | bolt://localhost:7687 desde host / bolt://neo4j:7687 dentro de Docker                     |
| Proposito            | Ejecutar consultas Cypher desde Python o desde cypher-shell para validar datos del grafo. |
| Autenticacion        | neo4j / <NEO4J_PASSWORD>.                                                                 |
| Headers recomendados | No aplica.                                                                                |

Cuerpo de solicitud:

| Campo  | Tipo   | Obligatorio | Descripción          | Ejemplo                               |
|--------|--------|-------------|----------------------|---------------------------------------|
| Cypher | string | Si          | Consulta a ejecutar. | MATCH (n) RETURN labels(n), count(n); |

Respuesta exitosa:

| Estado            | Contenido esperado                                   |
|-------------------|------------------------------------------------------|
| Resultado tabular | cypher-shell imprime filas con columnas solicitadas. |

Errores esperados:

| Estado                                | Causa probable                      | Accion sugerida                |
|---------------------------------------|-------------------------------------|--------------------------------|
| Connection refused                    | Puerto Bolt no disponible.          | Verificar mapeo 7687:7687.     |
| AuthenticationRateLimit               | Credenciales incorrectas repetidas. | Esperar o corregir contrasena. |
| Neo.ClientError.Statement.SyntaxError | Cypher mal escrito.                 | Validar sintaxis MATCH/RETURN. |

Ejemplo de consumo:

docker compose exec neo4j cypher-shell -u neo4j -p <NEO4J\_PASSWORD> "MATCH (n) RETURN labels(n) AS tipo, count(n) AS cantidad;"

### 17.6.3 Consultar conteo de nodos por etiqueta

| Campo                | Detalle                                                                           |
|----------------------|-----------------------------------------------------------------------------------|
| Metodo               | Cypher MATCH                                                                      |
| URL base / ruta      | Neo4j Bolt / cypher-shell                                                         |
| Proposito            | Demostrar que el grafo contiene usuarios, productos, ordenes y ubicacion central. |
| Autenticacion        | neo4j / <NEO4J_PASSWORD>.                                                         |
| Headers recomendados | No aplica.                                                                        |

Cuerpo de solicitud:

| Campo                      | Tipo       | Obligatorio | Descripcion                 | Ejemplo           |
|----------------------------|------------|-------------|-----------------------------|-------------------|
| MATCH (n)                  | pattern    | Si          | Selecciona todos los nodos. | MATCH (n)         |
| RETURN labels(n), count(n) | projection | Si          | Agrupar por etiqueta.       | labels(n) AS tipo |

Respuesta exitosa:

| Estado               | Contenido esperado                            |
|----------------------|-----------------------------------------------|
| Resultado verificado | User=100, Product=30, Order=2000, Location=1. |

Errores esperados:

| Estado        | Causa probable              | Accion sugerida                    |
|---------------|-----------------------------|------------------------------------|
| 0 rows        | El grafo no fue cargado.    | Ejecutar tarea cargar_grafo_neo4j. |
| labels vacios | Nodos creados sin etiqueta. | Revisar script de carga a Neo4j.   |

Ejemplo de consumo:

docker compose exec neo4j cypher-shell -u neo4j -p <NEO4J\_PASSWORD> "MATCH (n) RETURN labels(n) AS tipo, count(n) AS cantidad;"

### 17.6.4 Consultar conteo de relaciones por tipo

| Campo                | Detalle                                                                                    |
|----------------------|--------------------------------------------------------------------------------------------|
| Metodo               | Cypher MATCH                                                                               |
| URL base / ruta      | Neo4j Bolt / cypher-shell                                                                  |
| Proposito            | Validar que el modelo de grafo contiene relaciones MADE, CONTAINS, RECOMMENDED y ROUTE_TO. |
| Autenticacion        | neo4j / <NEO4J_PASSWORD>.                                                                  |
| Headers recomendados | No aplica.                                                                                 |

Cuerpo de solicitud:

| Campo | Tipo | Obligatorio | Descripcion | Ejemplo |
|-------|------|-------------|-------------|---------|
|-------|------|-------------|-------------|---------|

|                          |            |    |                                            |                     |
|--------------------------|------------|----|--------------------------------------------|---------------------|
| MATCH ()-[r]->()         | pattern    | Si | Selecciona todas las relaciones dirigidas. | MATCH ()-[r]->()    |
| RETURN type(r), count(r) | projection | Si | Agrupar por tipo de relacion.              | type(r) AS relacion |

Respuesta exitosa:

| Estado               | Contenido esperado                                     |
|----------------------|--------------------------------------------------------|
| Resultado verificado | MADE=2000, CONTAINS=4772, RECOMMENDED=41, ROUTE_TO=10. |

Errores esperados:

| Estado       | Causa probable               | Accion sugerida                                      |
|--------------|------------------------------|------------------------------------------------------|
| 0 relaciones | La carga de grafo no corrio. | Ejecutar cargar_grafo_neo4j y asignar_rutas_entrega. |

Ejemplo de consumo:

docker compose exec neo4j cypher-shell -u neo4j -p <NEO4J\_PASSWORD> "MATCH ()-[r]->() RETURN type(r) AS relacion, count(r) AS cantidad;"

### 17.6.5 Visualizar rutas en grafo

| Campo                | Detalle                                                                                     |
|----------------------|---------------------------------------------------------------------------------------------|
| Metodo               | Cypher MATCH path                                                                           |
| URL base / ruta      | Neo4j Browser / Bolt                                                                        |
| Proposito            | Mostrar graficamente relaciones entre nodos para evidenciar el uso de Neo4j en el proyecto. |
| Autenticacion        | neo4j / <NEO4J_PASSWORD>.                                                                   |
| Headers recomendados | No aplica.                                                                                  |

Cuerpo de solicitud:

| Campo              | Tipo    | Obligatorio | Descripcion                                               | Ejemplo            |
|--------------------|---------|-------------|-----------------------------------------------------------|--------------------|
| MATCH p=()-[r]->() | pattern | Si          | Obtiene caminos de un salto.                              | MATCH p=()-[r]->() |
| LIMIT              | integer | No          | Limita cantidad de caminos para que el grafo sea legible. | 50                 |

Respuesta exitosa:

| Estado           | Contenido esperado                        |
|------------------|-------------------------------------------|
| Resultado visual | Neo4j Browser muestra nodos y relaciones. |

Errores esperados:

| Estado            | Causa probable                         | Accion sugerida                               |
|-------------------|----------------------------------------|-----------------------------------------------|
| Grafo muy cargado | La consulta devuelve demasiados nodos. | Usar LIMIT 50 o filtrar por tipo de relacion. |

Ejemplo de consumo:

MATCH p=()-[r]->() RETURN p LIMIT 50;

## 17.7 Spark / PySpark - ejecucion del analisis

Spark no se expone como API REST de negocio. En el proyecto se ejecuta desde Airflow mediante Python, y se valida con la interfaz del Spark Master y con los archivos generados.

### 17.7.1 Interfaz Spark Master

| Campo                | Detalle                                                                            |
|----------------------|------------------------------------------------------------------------------------|
| Metodo               | GET                                                                                |
| URL base / ruta      | http://localhost:8081                                                              |
| Proposito            | Verificar que el master Spark esta disponible y que los workers pueden conectarse. |
| Autenticacion        | Sin autenticacion en ambiente local.                                               |
| Headers recomendados | No requiere headers.                                                               |

Respuesta exitosa:

| Estado | Contenido esperado        |
|--------|---------------------------|
| 200 OK | Carga UI de Spark Master. |

Errores esperados:

| Estado             | Causa probable               | Accion sugerida                     |
|--------------------|------------------------------|-------------------------------------|
| Connection refused | spark-master no esta activo. | docker compose ps spark-master.     |
| Puerto ocupado     | Otro servicio usa 8081.      | Cambiar mapeo de puerto en compose. |

Ejemplo de consumo:

http://localhost:8081

### 17.7.2 Ejecutar script de analisis Spark

| Campo                | Detalle                                                                                    |
|----------------------|--------------------------------------------------------------------------------------------|
| Metodo               | Python CLI dentro de Airflow                                                               |
| URL base / ruta      | /opt/airflow/proyecto_olap/spark_analisis.py                                               |
| Proposito            | Procesar orders, order_details, products y users para producir tres resultados analiticos. |
| Autenticacion        | No aplica. Se ejecuta dentro del contenedor airflow.                                       |
| Headers recomendados | No aplica.                                                                                 |

Cuerpo de solicitud:

| Campo         | Tipo       | Obligatorio | Descripcion                      | Ejemplo       |
|---------------|------------|-------------|----------------------------------|---------------|
| orders        | tabla JDBC | Si          | Tabla de pedidos.                | orders        |
| order_details | tabla JDBC | Si          | Detalle de productos por pedido. | order_details |
| products      | tabla JDBC | Si          | Catalogo de productos.           | products      |
| users         | tabla JDBC | Si          | Cientes.                         | users         |

Respuesta exitosa:

| Estado          | Contenido esperado                                                       |
|-----------------|--------------------------------------------------------------------------|
| Proceso exitoso | Muestra analisis 1, 2 y 3; termina con Analisis completado exitosamente. |

Errores esperados:

| Estado                       | Causa probable                          | Accion sugerida                          |
|------------------------------|-----------------------------------------|------------------------------------------|
| ModuleNotFoundError: pyspark | Imagen de Airflow no fue construida con | docker compose build --no-cache airflow. |

|                                               |                      |                                          |
|-----------------------------------------------|----------------------|------------------------------------------|
|                                               | Dockerfile.          |                                          |
| ClassNotFoundException: org.postgresql.Driver | Driver JDBC ausente. | Confirmar drivers/postgresql-42.7.3.jar. |
| relation orders does not exist                | Datos no generados.  | Ejecutar DAG completo.                   |

### Ejemplo de consumo:

`docker compose exec airflow python /opt/airflow/proyecto_olap/spark_analisis.py`

#### 17.7.3 Consultar carpeta de resultados Spark

| Campo                | Detalle                                           |
|----------------------|---------------------------------------------------|
| Metodo               | Filesystem dentro del contenedor Airflow          |
| URL base / ruta      | /opt/airflow/proyecto_olap/resultados_spark       |
| Proposito            | Verificar que Spark genere las salidas esperadas. |
| Autenticacion        | No aplica.                                        |
| Headers recomendados | No aplica.                                        |

### Respuesta exitosa:

| Estado              | Contenido esperado                                     |
|---------------------|--------------------------------------------------------|
| Listado de carpetas | crecimiento_mensual, horarios_pico y top_producto_mes. |

### Errores esperados:

| Estado                    | Causa probable                      | Accion sugerida                |
|---------------------------|-------------------------------------|--------------------------------|
| No such file or directory | Spark no ejecuto o ruta incorrecta. | Ejecutar spark_analisis.py.    |
| Carpetas vacias           | Fallo al escribir resultados.       | Revisar logs del script Spark. |

### Ejemplo de consumo:

`docker compose exec airflow ls /opt/airflow/proyecto_olap/resultados_spark`

## 17.8 Metabase - endpoint de visualizacion BI

#### 17.8.1 Abrir Metabase

| Campo                | Detalle                                                                                 |
|----------------------|-----------------------------------------------------------------------------------------|
| Metodo               | GET                                                                                     |
| URL base / ruta      | http://localhost:3000                                                                   |
| Proposito            | Acceder a Metabase para crear consultas, preguntas y dashboards sobre PostgreSQL.       |
| Autenticacion        | Usuario configurado en primer ingreso de Metabase. No forma parte automatizada del DAG. |
| Headers recomendados | No requiere headers desde navegador.                                                    |

### Respuesta exitosa:

| Estado | Contenido esperado             |
|--------|--------------------------------|
| 200 OK | Carga la pantalla de Metabase. |

### Errores esperados:

| Estado             | Causa probable                         | Accion sugerida                                |
|--------------------|----------------------------------------|------------------------------------------------|
| Connection refused | Contenedor metabase no activo.         | docker compose ps metabase.                    |
| Setup pendiente    | Metabase se abre en asistente inicial. | Completar configuracion y conectar PostgreSQL. |



Ejemplo de consumo:

http://localhost:3000

### 17.8.2 Conexion de Metabase a PostgreSQL

| Campo                | Detalle                                                                      |
|----------------------|------------------------------------------------------------------------------|
| Metodo               | JDBC interno Metabase -> PostgreSQL                                          |
| URL base / ruta      | Host: postgres, Puerto: 5432, BD: restaurantes                               |
| Proposito            | Permitir que Metabase consulte las vistas OLAP para generar visualizaciones. |
| Autenticacion        | Usuario admin de PostgreSQL.                                                 |
| Headers recomendados | No aplica.                                                                   |

Cuerpo de solicitud:

| Campo    | Tipo    | Obligatorio | Descripcion                                 | Ejemplo      |
|----------|---------|-------------|---------------------------------------------|--------------|
| Host     | string  | Si          | Nombre del servicio Docker para PostgreSQL. | postgres     |
| Port     | integer | Si          | Puerto interno de PostgreSQL.               | 5432         |
| Database | string  | Si          | Nombre de la base.                          | restaurantes |
| User     | string  | Si          | Usuario de base de datos.                   | admin        |

Respuesta exitosa:

| Estado           | Contenido esperado                                        |
|------------------|-----------------------------------------------------------|
| Conexion exitosa | Metabase muestra tablas y permite ejecutar consultas SQL. |

Errores esperados:

| Estado                | Causa probable                         | Accion sugerida                                      |
|-----------------------|----------------------------------------|------------------------------------------------------|
| Connection refused    | Metabase no puede llegar a PostgreSQL. | Usar host postgres, no localhost, dentro de Docker.  |
| Authentication failed | Usuario/contrasena incorrectos.        | Revisar variables POSTGRES_USER y POSTGRES_PASSWORD. |

Ejemplo de consumo:

# Valores para el asistente de Metabase

Database type: PostgreSQL

Host: postgres

Port: 5432

Database name: restaurantes

Username: admin

Password: <POSTGRES\_PASSWORD>

### 17.9 Mapa de endpoint a evidencia

| Endpoint / integracion | Comando o pantalla          | Captura recomendada                   | Resultado esperado                |
|------------------------|-----------------------------|---------------------------------------|-----------------------------------|
| Docker Compose         | docker compose ps           | 01_docker_compose_ps.png              | Servicios en estado Up.           |
| PostgreSQL tablas      | information_schema.tables   | 02_postgres_tablas_vistas.png         | 19 tablas/vistas.                 |
| PostgreSQL datos       | SELECT COUNT(*) FROM orders | 03_postgres_conteo_orders.png         | 2000.                             |
| Spark analisis         | spark_analisis.py           | 04 y 05_spark_analisis*.png           | Analisis completado exitosamente. |
| Elasticsearch count    | GET /productos/_count       | 06_elasticsearch_conteo_productos.png | count = 30.                       |
| Airflow Graph          | http://localhost:8080       | 07_airflow_dag_graph_success.png      | Todas las tareas en verde.        |

|                      |                        |                                        |                                        |
|----------------------|------------------------|----------------------------------------|----------------------------------------|
| Neo4j nodos          | MATCH (n)              | 08_neo4j_conteo_nodos.png              | User, Product, Order, Location.        |
| Neo4j relaciones     | MATCH ()-[r]->()       | 09_neo4j_conteo_relaciones.png         | MADE, CONTAINS, RECOMMENDED, ROUTE_TO. |
| Resultados Spark     | ls resultados_spark    | 10_spark_resultados_guardados.png      | 3 carpetas de salida.                  |
| Neo4j Browser        | MATCH p=()-[r]->()     | 11_neo4j_browser_grafo_visual.png      | Grafo visual.                          |
| Elasticsearch search | GET /productos/_search | 12_elasticsearch_busqueda_producto.png | Producto_1 encontrado.                 |

## 17.10 Consideraciones de seguridad y buenas practicas para endpoints

| Riesgo                      | Situacion en el proyecto                                                     | Recomendacion                                                                             |
|-----------------------------|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Credenciales visibles       | Las contraseñas se usaron en comandos locales.                               | No subir capturas con cat standalone_admin_password.txt ni exponer contraseñas en README. |
| Elasticsearch sin seguridad | xpack.security.enabled=false para entorno local.                             | Aclarar que es ambiente academico; en produccion habilitar seguridad y TLS.               |
| Puertos expuestos           | Servicios disponibles en localhost.                                          | Evitar exponer puertos a redes publicas.                                                  |
| Datos duplicados            | Ejecutar el DAG varias veces puede duplicar datos si los scripts no limpian. | Antes de repetir pruebas, confirmar si las tablas se limpian o reiniciar volumen.         |
| Uso de contraseñas en shell | Los comandos quedan en historial.                                            | Usar placeholders en documentacion y evitar subir historial/terminal con credenciales.    |

## 17.11 Referencias usadas para la estructura de documentacion de endpoints

- **Spotify Web API - Search endpoint reference:**  
<https://developer.spotify.com/documentation/web-api/reference/search>
- **OpenAPI Specification - standard interface for HTTP APIs:** <https://swagger.io/specification/>
- **Apache Airflow 2.8 Stable REST API reference:**  
<https://airflow.apache.org/docs/apache-airflow/2.8.4/stable-rest-api-ref.html>
- **Elasticsearch Search API:** <https://www.elastic.co/docs/api/doc/elasticsearch/operation/operation-search>
- **Elasticsearch Count API:** <https://www.elastic.co/docs/api/doc/elasticsearch/operation/operation-count>
- **Neo4j Cypher MATCH clause:** <https://neo4j.com/docs/cypher-manual/current/clauses/match/>
- **PostgreSQL psql documentation:** <https://www.postgresql.org/docs/current/app-psql.html>
- **Metabase SQL editor documentation:** <https://www.metabase.com/docs/latest/questions/native-editor/writing-sql>